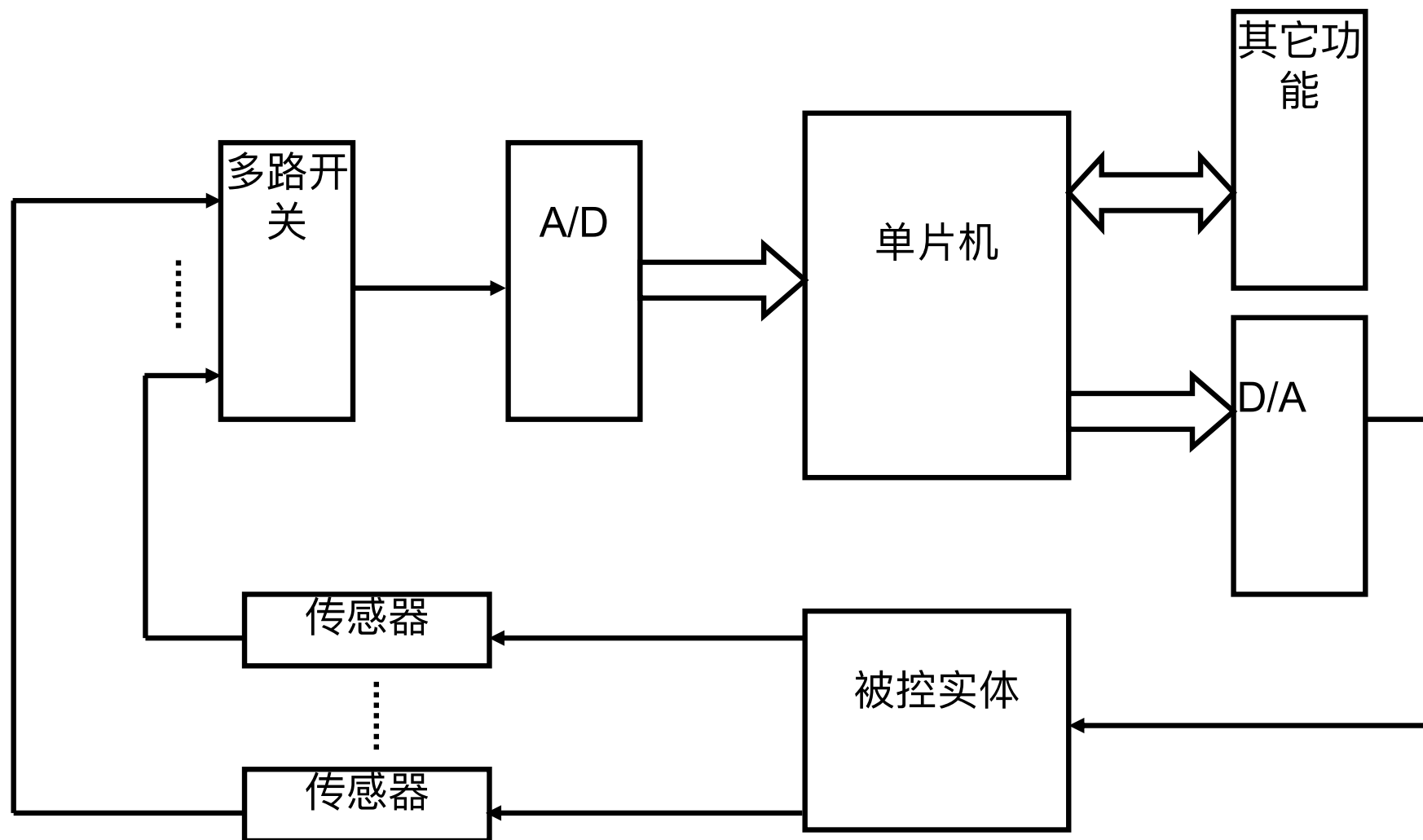


《单片机原理及应用》 第十二讲： 模数转换系统



- **A/D转换器**，或简写ADC（Analog to Digital Converter）是一种能把模拟量转换成相应的数字量的电子器件。
- **D/A转换器**，或简写DAC（Digital To Analog Converter）则相反，它能把数字量转换成相应的模拟量。





12.1 模/数转换器

- A/D是将模拟量转换成数字量的器件。
- 模拟量可以是电压、电流等电信号，也可以是声、光、压力、湿度、温度等随时间连续变化的非电的物理量。非电的模拟量可通过合适的传感器（如光电传感器、压力传感器、温度传感器）转换成电信号。
- C8051F020片内包含一个9通道的12位的模数转换器**ADC0**和8通道8位的模数转换器**ADC1**。



- 1.转换原理

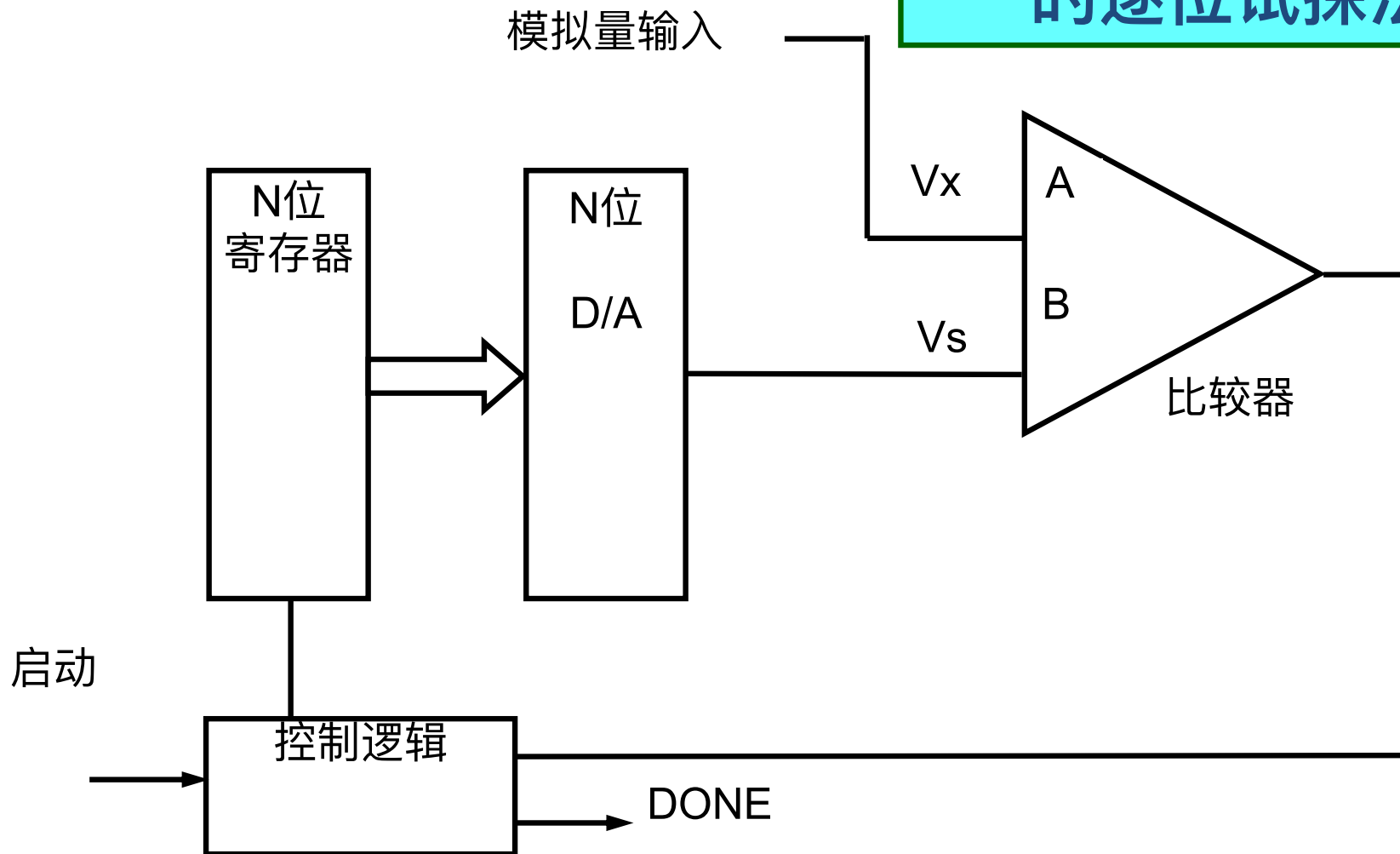
A/D转换器的种类很多，根据转换原理可以分

- ▶ 计数式
- ▶ 平行式
- ▶ 双积分式
- ▶ 逐次逼近式
- ▶ ...



逐次逼近式A/D转换器

从最高位开始的逐位试探法

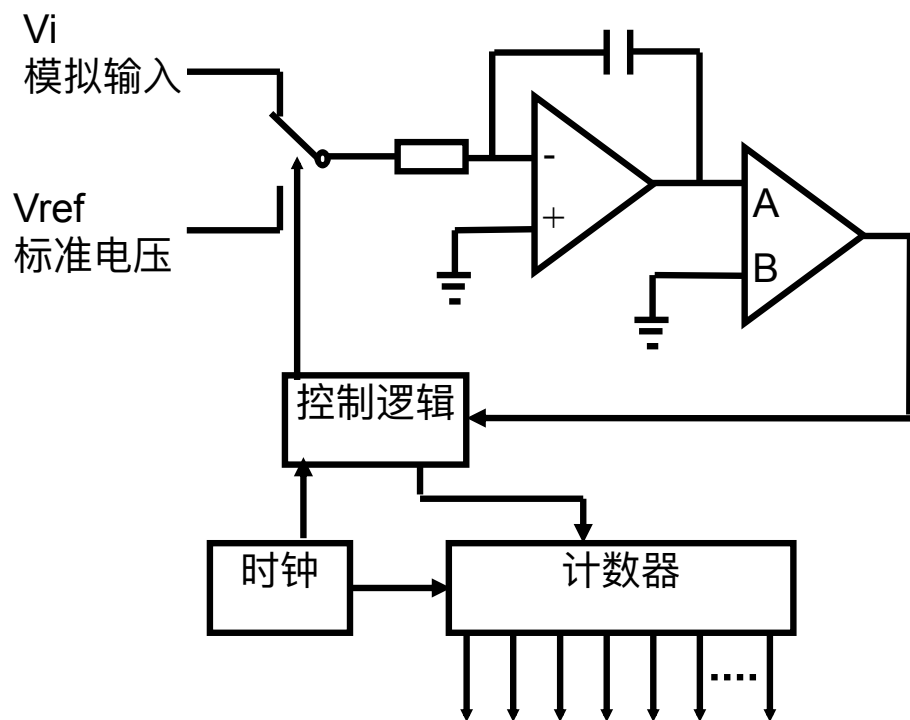


采用对分搜索法逐次比较、逐步逼近的原理来转换

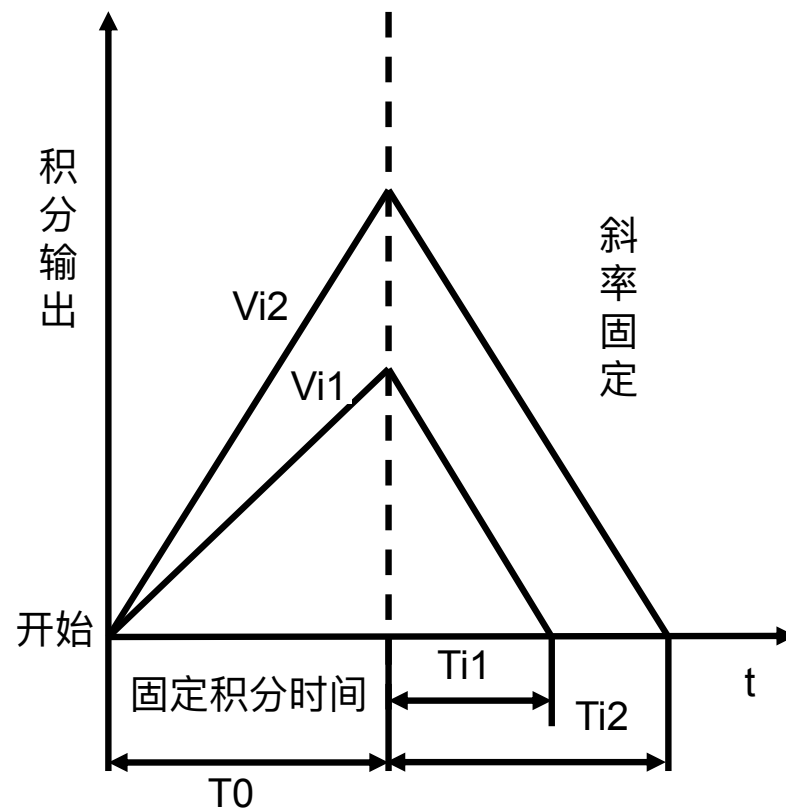
- ▶ 控制逻辑先置1结果寄存器最高位 D_{n-1}
- ▶ 若 $V_x >$ 经转换的 V_s 则保留此位 D_{n-1} （为1），否则清“0” D_{n-1} 位。
- ▶ 然后控制逻辑置1结果寄存器次高位 D_{n-2}
- ▶ V_s 再和 V_x 比较，以决定 D_{n-2} 位保留为1还是清成“0”，依次类推到最低位 D_0 。
- ▶ 结果寄存器的状态便是与输入的模拟量 V_x 对应的数字量。



双积分式A/D转换器



(a) 原理框图



(b) 积分原理



- 双积分式也称二重积分式，其实质是测量和比较两个积分的时间。
- 一个是对模拟输入电压积分的时间 T_0 ，此时间往往是固定的；另一个是以充电后的电压为初值，对参考电源 V_{ref} 反向积分，积分电容被放电至零所需的时间 T_i (V_{ref} 与 V_i 符号相反)。模拟输入电压 V_i 与参考电压 V_{ref} 之比，等于上述两个时间之比。由于 V_{ref} 、 T_0 固定，而放电时间 T_i 可以测出，因而可计算出模拟输入电压的大小。
- $V_i/V_{ref} = T_0/T_i \Rightarrow V_i = T_0/T_i * V_{ref}$
- 转换速度较慢，为ms级
- 抗干扰能力强
- 适用于精度要求高，现场干扰较严重，信号变化缓慢的场合



2. 性能指标

- 衡量A/D性能的主要参数是：
 - (1) 分辨率 (resolution)
- 分辨率是指输出的数字量变化一个相邻的值所对应的输入模拟量的变化值；取决于输出数字量的二进制位数。
- 一个n位的A/D转换器所能分辨的最小输入模拟增量定义为满量程值的 2^{-n} 倍。
- 例如，满量程为10V的8位A/D芯片的分辨率为 $10V \times 2^{-8} = 39\text{mV}$ ；
- 而16位的A/D是 $10V \times 2^{-16} = 153\mu\text{V}$ 。



- (2) 满刻度误差 (full scale error)

满刻度误差也称增益误差，即输出全1时输入电压与理想输入量之差。

- (3) 转换速率 (conversion rate)

转换速度是指完成一次A/D转换所需时间的倒数，A/D转换器型号不同，转换速度差别很大。

- (4) 转换精度 (conversion accuracy)

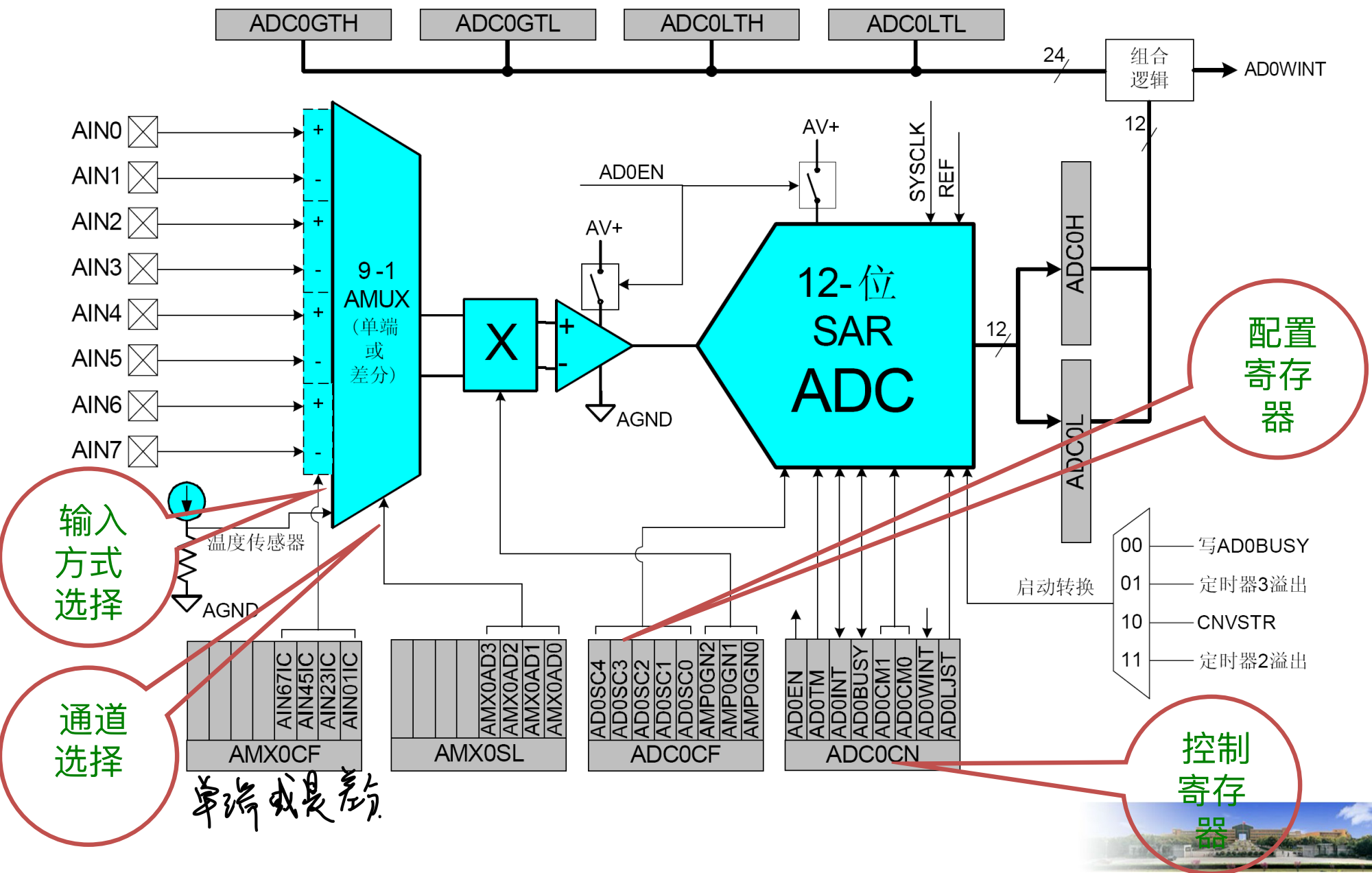
转换精度由模拟误差和数字误差组成。模拟误差属于非固定误差，由器件质量决定，数字误差和A/D输出数字量的位数有关，位数越多，误差越小。



- C8051F020的ADC0子系统就是一个100ksps、12 位分辨率的逐次逼近寄存器型ADC。
- 最高转换速度为100ksps，转换时钟来源于系统时钟分频，分频值保存在寄存器ADC0CF的ADCSC位。
- 它包括一个9通道的可编程模拟多路选择器（AMUX0），一个可编程增益放大器（PGA0）和一个100ksps、12 位分辨率的逐次逼近寄存器型ADC，ADC中集成了跟踪保持电路和可编程窗口检测器。



ADC0子系统功能框图



- ADC0的运行主要与图上标的10个SFR有关。
- 8个外部输入的模拟量可以通过**通道配置寄存器AMX0CF**设定为单端输入或双端输入；
- 8个外部输入的模拟量和一个内部温度传感器量通过**通道选择寄存器AMX0SL**设定在某一时刻通过多路选择器的模拟量；
- 通过**ADC配置寄存器ADC0CF**设定ADC转换速度和对模拟量的放大倍数；
- 由**控制寄存器ADC0CN**对ADC进行模拟量转换的启动、启动方式、采样保持、转换结束、数字量格式等进行设定；
- 12位的转换好的数字量存放在**数据字寄存器ADC0H、ADC0L**中；



- ADC0中提供了可编程窗口检测器，通过**上下限寄存器ADC0GTH、ADC0GTL、ADC0LTH、ADC0LTL**设定所需要的比较极限值。
- 在进行模拟量转换前设定好以上SFR，CPU就按设定好的模式在模拟量转换好时用指令读出数据寄存器中的数字量或在中断服务程序中读取数字量。然后再进行下一次的转换。



12.3 模拟多路选择器和PGA

- 模拟多路选择器AMUX (Analog Multiplexer) 中的8个通道用于外部测量，而第九通道在内部被接到片内温度传感器。
- 寄存器AMX0CF进行将输入对编程为差分或单端方式；寄存器AMX0SL进行通道选择。



• 配置寄存器AMX0CF的格式如下：

R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	复位值
—	—	—	—	AIN67IC	AIN45IC	AIN23IC	AIN01IC	00000000
位7	位6	位5	位4	位3	位2	位1	位0	SFR地址： 0xBA

位7~位4：不使用，读0000b，写忽略

位3： AIN67IC： AIN6、 AIN7 输入对配置位

0: AIN6 和AIN7 为独立的单端输入

1: AIN6, AIN7 为+, -差分输入对

位2~位0类推。



• 通道选择寄存器AMX0SL的格式如下：

R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	复位值
—	—	—	—	AMX0AD3	AMX0AD2	AMX0AD1	AMX0AD0	00000000
位7	位6	位5	位4	位3	位2	位1	位0	SFR地址： 0xBB

位7~位4：不使用，读0000b，写忽略

位3-0： AMX0AD3-0: AMUX0 地址位

0000-1111b: 根据不同组合选择ADC 输入的通道。





		AMX0AD3-0								
		0000	0001	0010	0011	0100	0101	0110	0111	1xxx
AMX0CF 位 3-0	0000	AIN0	AIN1	AIN2	AIN3	AIN4	AIN5	AIN6	AIN7	温度传感器
	0001	+(AIN0) -(AIN1)		AIN2	AIN3	AIN4	AIN5	AIN6	AIN7	温度传感器
	0010	AIN0	AIN1	+(AIN2) -(AIN3)		AIN4	AIN5	AIN6	AIN7	温度传感器
	0011	+(AIN0) -(AIN1)		+(AIN2) -(AIN3)		AIN4	AIN5	AIN6	AIN7	温度传感器
	0100	AIN0	AIN1	AIN2	AIN3	+(AIN4) -(AIN5)		AIN6	AIN7	温度传感器
	0101	+(AIN0) -(AIN1)		AIN2	AIN3	+(AIN4) -(AIN5)		AIN6	AIN7	温度传感器
	0110	AIN0	AIN1	+(AIN2) -(AIN3)		+(AIN4) -(AIN5)		AIN6	AIN7	温度传感器
	0111	+(AIN0) -(AIN1)		+(AIN2) -(AIN3)		+(AIN4) -(AIN5)		AIN6	AIN7	温度传感器
	1000	AIN0	AIN1	AIN2	AIN3	AIN4	AIN5	+(AIN6) -(AIN7)		温度传感器
	1001	+(AIN0) -(AIN1)		AIN2	AIN3	AIN4	AIN5	+(AIN6) -(AIN7)		温度传感器
	1010	AIN0	AIN1	+(AIN2) -(AIN3)		AIN4	AIN5	+(AIN6) -(AIN7)		温度传感器
	1011	+(AIN0) -(AIN1)		+(AIN2) -(AIN3)		AIN4	AIN5	+(AIN6) -(AIN7)		温度传感器
	1100	AIN0	AIN1	AIN2	AIN3	+(AIN4) -(AIN5)		+(AIN6) -(AIN7)		温度传感器
	1101	+(AIN0) -(AIN1)		AIN2	AIN3	+(AIN4) -(AIN5)		+(AIN6) -(AIN7)		温度传感器
	1110	AIN0	AIN1	+(AIN2) -(AIN3)		+(AIN4) -(AIN5)		+(AIN6) -(AIN7)		温度传感器
	1111	+(AIN0) -(AIN1)		+(AIN2) -(AIN3)		+(AIN4) -(AIN5)		+(AIN6) -(AIN7)		温度传感器



- PGA (programmable gain amplifier), 可编程增益放大器.
- 它对AMUX 输出信号的放大倍数由ADC0 配置寄存器 ADC0CF中的AMP0GN2-0 确定。
- PGA 增益可以用软件编程为0.5、1、2、4、8 或16，复位后的默认增益为1。
- 注意，PGA0 的增益对温度传感器也起作用。
- 配置寄存器ADC0CF的格式如下：

R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	复位值
AD0SC4	AD0SC3	AD0SC2	AD0SC1	AD0SC0	AMP0GN2	AMP0GN1	AMP0GN0	11111000
位7	位6	位5	位4	位3	位2	位1	位0	SFR地址: 0xBC

位7~3: AD0SC4~0, ADC0SAR转换时钟控制位, CLK_{SAR0} 表示ADC0 SAR时钟

$$AD0SC = \frac{SYSCLK}{CLK_{SAR0}} - 1$$

全0, 转换时钟最快为SYSCLK.

位2~0: AMP0GN2~0, ADC0内部放大器增益 (PGA)

000: 增益=1, 001: 增益=2, 010: 增益=4

011: 增益=8, 10X: 增益=16, 11X: 增益=0.5



12.4 ADC的工作方式

- 1. 转换过程
- ADC0的转换过程由控制寄存器ADC0CN来设置
- 控制寄存器ADC0CN的格式如下：

R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	复位值
AD0EN	AD0TM	AD0INT	AD0BUSY	AD0CM1	AD0CM0	AD0WINT	AD0LJST	00000000
位7	位6	位5	位4	位3	位2	位1	位0 (可位寻址)	SFR地址: 0xE8



控制寄存器ADC0CN

R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	复位值
AD0EN	AD0TM	AD0INT	AD0BUSY	AD0CM1	AD0CM0	AD0WINT	AD0LJST	00000000
位7	位6	位5	位4	位3	位2	位1	位0	SFR地址: 0xE8
							(可位寻址)	

位7: AD0EN, ADC0使能位, 0禁止, 1使能

位6: AD0TM, ADC0跟踪方式位。

0: ADC0被使能时, 除了转换期间外一直处于跟踪方式

1: 由AD0CM1~0定义跟踪方式

位5: AD0INT, ADC0转换结束**中断标志**。该标志必须用软件清0

0: 最后一次清0后, 还没有完成一次数据转换

1: ADC0完成了一次数据转换

位4: AD0BUSY, ADC0忙标志位

读 0: ADC0转换结束或者当前没有正在进行的数据转换

1: ADC0正在进行数据转换

写 0: 无作用, 1: 若AD0CM1,0=00, 启动ADC0转换



控制寄存器ADC0CN

R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	复位值
AD0EN	AD0TM	AD0INT	AD0BUSY	AD0CM1	AD0CM0	AD0WINT	AD0LJST	00000000
位7	位6	位5	位4	位3	位2	位1	位0 (可位寻址)	SFR地址: 0xE8

位3,2: AD0CM1,0, ADC0启动方式选择位

如果AD0TM=0

- 00: 向AD0BUSY写1启动ADC0转换
- 01: 定时器3溢出启动ADC0转换
- 10: CNVSTR上升沿启动ADC0转换
- 11: 定时器2溢出启动ADC0转换

如果AD0TM=1

- 00: 向AD0BUSY写1启动跟踪, 持续3个SAR时钟后进行转换
- 01: 定时器3溢出启动跟踪, 持续3个SAR时钟后进行转换
- 10: 只有当CNVSTR输入为逻辑低电平时ADC0跟踪, 在CNVSTR的上升沿开始转换
- 11: 定时器2溢出启动跟踪, 持续3个SAR时钟后进行转换



控制寄存器ADC0CN

R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	复位值
AD0EN	AD0TM	AD0INT	AD0BUSY	AD0CM1	AD0CM0	AD0WINT	AD0LJST	00000000
位7	位6	位5	位4	位3	位2	位1	位0 (可位寻址)	SFR地址: 0xE8

位1: AD0WINT, ADC0比较中断标志, 该位必须软件清0

0: 该位被清0后未发生过ADC0窗口比较匹配

1: 发生了ADC0窗口比较匹配

位0: AD0LJST, ADC0数据左或右对齐选择位

0: ADC0H: ADC0L寄存器数据右对齐

1: ADC0H: ADC0L寄存器数据左对齐



- C8051F020的ADC0有4种转换启动方式，由ADC0CN 中的ADC0 启动转换方式位（AD0CM1，AD0CM0）的状态决定。
 - 00：向AD0BUSY 写1 启动ADC0 转换。
 - 01：定时器3 溢出启动ADC0 转换。
 - 10：CNVSTR 上升沿启动ADC0 转换。
 - 11：定时器2 溢出启动ADC0 转换。
- AD0BUSY 位在转换期间被置‘1’，转换结束后复‘0’
- AD0BUSY位的下降沿触发一个中断，并将中断标志AD0INT置‘1’
- 转换数据被保存在ADC数据字的MSB和LSB寄存器对ADC0H和ADC0L中
- 转换数据在寄存器对ADC0H：ADC0L中的存储方式可以是左对齐或者右对齐，由ADC0CN寄存器中AD0LJST位的编程状态决定



- 当通过向AD0BUSY写‘1’启动数据转换时，应查询AD0INT位以确定转换何时结束。
- 建议的查询步骤如下：
 - (1) 写‘0’到AD0INT；（清零中断标志）
 - (2) 向AD0BUSY 写‘1’；（启动转换）
 - (3) 查询并等待AD0INT 变‘1’；（等待转换结束）
 - (4) 处理ADC0 数据；



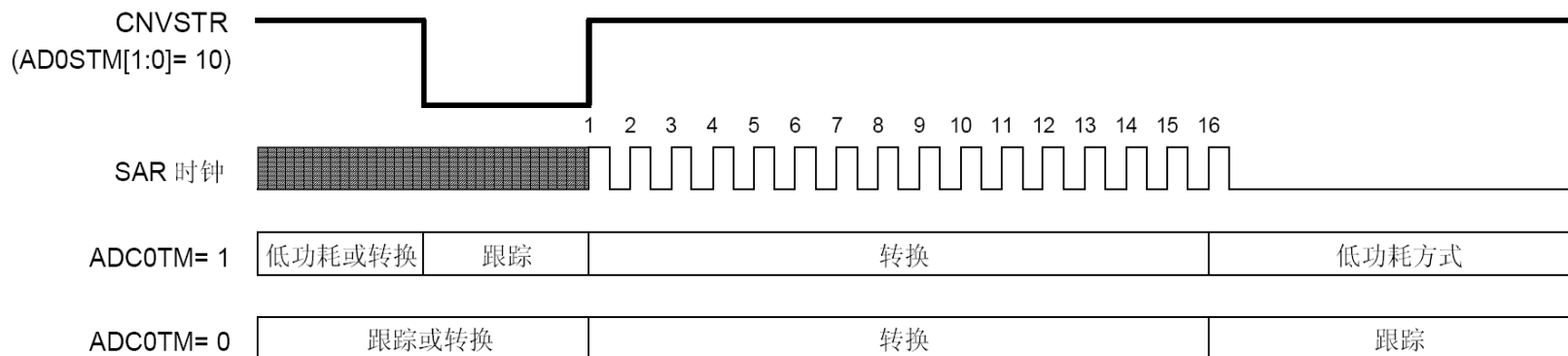
2. 跟踪方式

- ADC0CN 中的AD0TM 位控制ADC0 的跟踪保持方式。
- 在缺省状态，除了转换期间之外ADC0 输入被连续跟踪。
- 当AD0TM =1，ADC0 工作在低功耗跟踪保持方式。每次转换之前都有3个SAR 时钟的跟踪周期（在启动转换信号有效之后）。
- 当CNVSTR 信号用于在低功耗跟踪保持方式启动转换时，ADC0 只在CNVSTR 为低电平时跟踪；在CNVSTR 的上升沿开始转换。
- 当整个芯片处于低功耗待机或休眠方式时，跟踪可以被禁止（关断）。

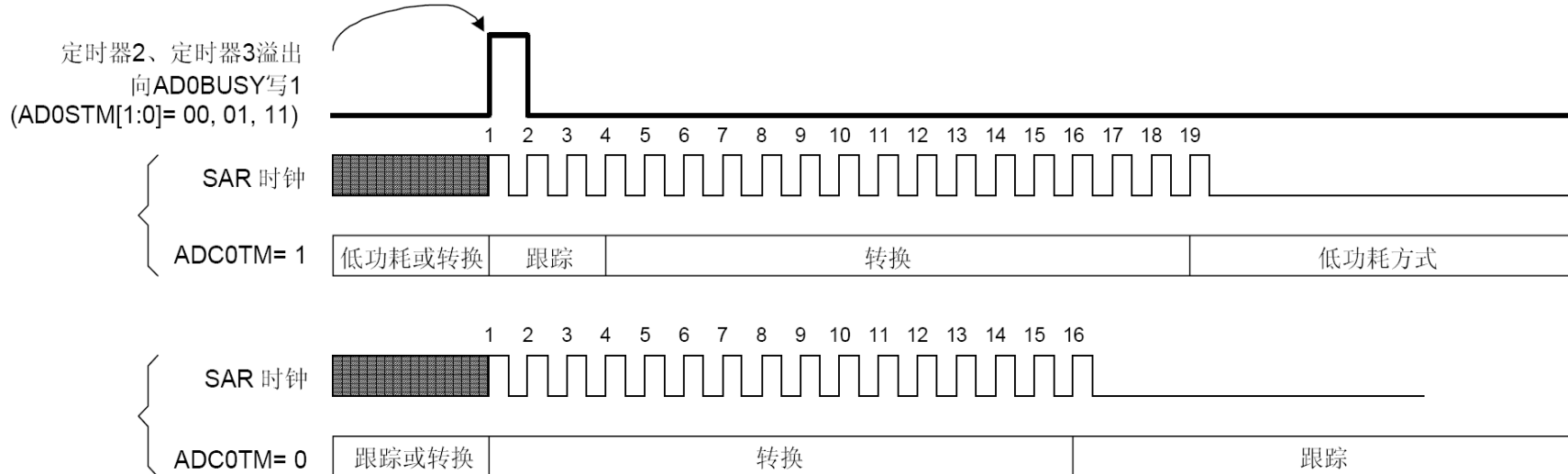


12位ADC 跟踪和转换时序

A. 使用外部触发源的ADC时序



B. 使用内部触发源的ADC时序



3. 建立时间要求

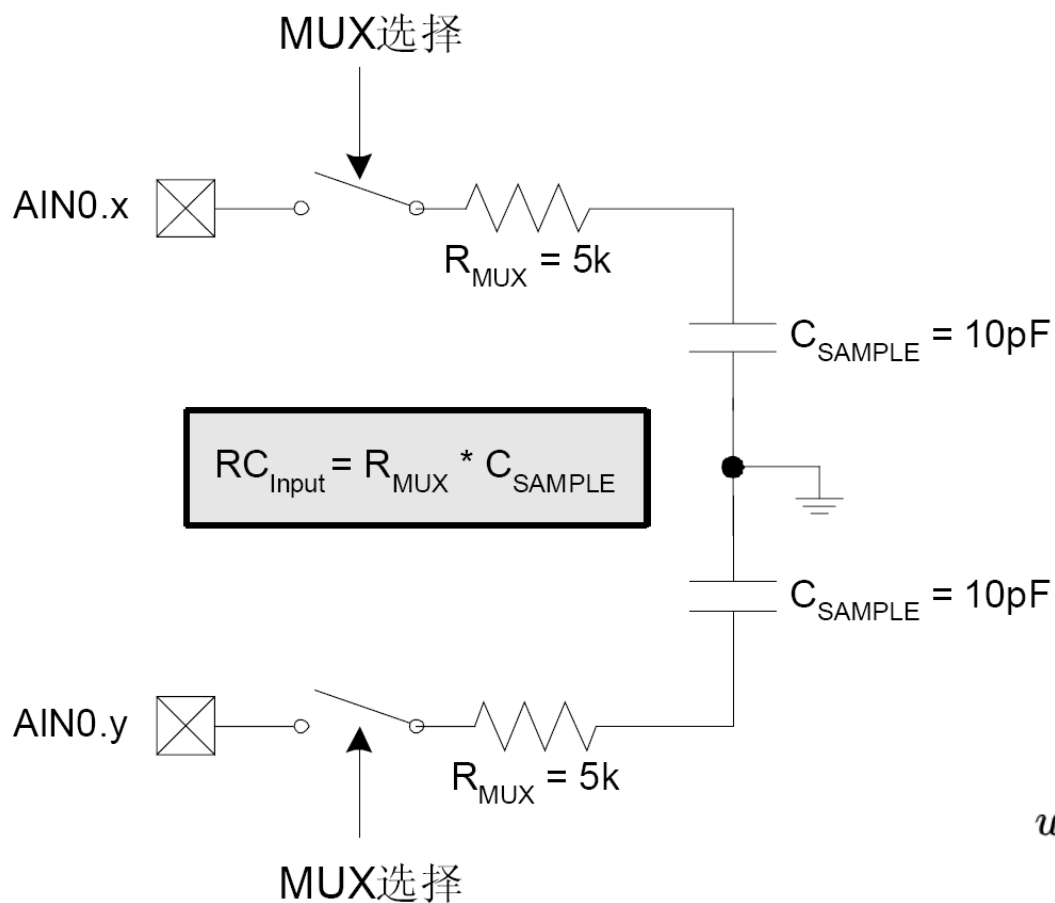
- 当ADC0 输入配置发生改变时（AMUX 或PGA 的选择发生变化），**在进行一次精确的转换之前需要有一个最小的跟踪时间。**
- 最小跟踪时间由ADC0模拟多路器的电阻、ADC0采样电容、外部信号源阻抗以及所要求的转换精度决定
- 对于一个给定的建立精度（ SA ），所需要的ADC0 建立时间可以用方程估算：

$$t = \ln \left[\frac{2^n}{SA} \right] \times R_{TOTAL} C_{SAMPLE}$$

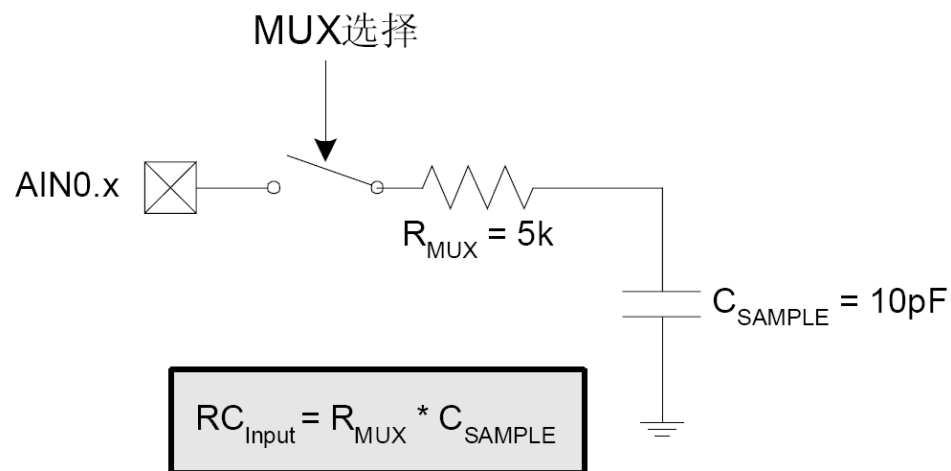
- SA 为建立精度，用一个LSB的分数来表示。 t 为建立时间，以秒为单位。 R_{TOTAL} 为ADC0 模拟多路器电阻与外部信号源电阻之和； n 为ADC0 的分辨率，用比特表示。



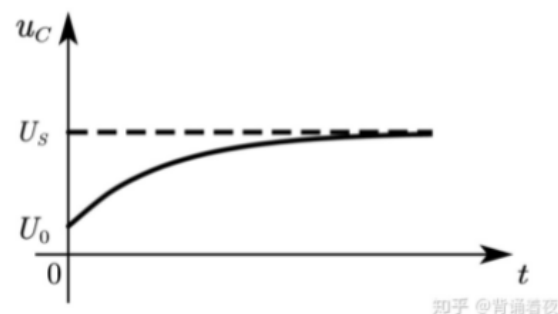
ADC0 等效输入电路



差分



单端



$$u_C(t) = U_S + (U_0 - U_S) e^{-\frac{1}{RC}t} \quad t \geq 0$$



4. 转换结果格式

- 12位的转换好的数字量存放在数据字寄存器ADC0H、ADC0L中。这是两个8位寄存器。
- **当AD0LJST=0：即寄存器数据右对齐**，ADC0H的位7-4为位3的符号扩展位。位3-0是12位ADC0数据字的高4位。ADC0L的位7-0为12位ADC0数据字的低8位。
- **当AD0LJST=1：即寄存器数据左对齐**，ADC0H位7-0为12位ADC0数据字的高8位。ADC0L的位7-4为12位ADC0数据字的低4位。位3-0总是0。



数据字转换表

- AIN0 为单端输入方式：

(AMX0CF=0x00, AMX0SL=0x00)

AIN0-AGND(伏)	ADC0H:ADC0L (AD0LJST=0)	ADC0H:ADC0L (AD0LJST=1)
VREF * (4095/4096)	0x0FFF	0xFFFF0
VREF / 2	0x0800	0x8000
VREF * (2047/4096)	0x07FF	0x7FF0
0	0x0000	0x0000

- 数据是无符号数



AIN0-AIN1 为差分输入对:

(AMX0CF=0x01, AMX0SL=0x00)

AIN0-AIN1(伏)	ADC0H:ADC0L (AD0LJST=0)	ADC0H:ADC0L (AD0LJST=1)
$V_{REF} * (2047/2048)$	0x07FF	0x7FF0
$V_{REF} / 2$	0x0400	0x4000
$V_{REF} \times (1/2048)$	0x0001	0x0010
0	0x0000	0x0000
$-V_{REF} \times (1/2048)$	0xFFFF (-1d)	0xFFFF0
$-V_{REF} / 2$	0xFC00 (-1024d)	0xC000
$-V_{REF}$	0xF800 (-2048d)	0x8000

- 数据为带符号数 (右对齐, 高四位为符号扩展)



12.5 ADC0 可编程窗口检测器

- ADC0 可编程窗口检测器提供一个中断，当ADC0转换值在ADC0下限（大于）寄存器ADC0GTH: ADC0GTL和ADC0上限（小于）寄存器ADC0LTH: ADC0LTL范围之内，并且中断开启时，引发相应中断。
- 只要 $ADC0 > ADC0GTH: ADC0GTL$ 或者（并且） $ADC0 < ADC0LTH: ADC0LTL$ 就可引起中断。
- 可应用于节能场合，如让系统处于空闲状态，当ADC0输入信号在窗口检测器预设值的范围内时，引发中断，唤醒CIP-51进行相应的处理
- 窗口检测器中断标志（ADC0CN的AD0WINT位）也可被用于查询方式。



ADC0 右对齐的单端数据窗口中断示例

数据在此范围中断

输入电压 (AD0-AGND)	ADC0 数据字	
$REF \times (4095/4096)$	0x0FFF	不影响 AD0WINT
	0x0201	
$REF \times (512/4096)$	0x0200	ADC0LTH:ADC0LTL
	0x01FF	AD0WINT=1
	0x0101	
$REF \times (256/4096)$	0x0100	ADC0GTH:ADC0GTL
	0x00FF	不影响 AD0WINT
0	0x0000	

输入电压 (AD0-AGND)	ADC0 数据字	
$REF \times (4095/4096)$	0x0FFF	AD0WINT=1
	0x0201	
$REF \times (512/4096)$	0x0200	ADC0GTH:ADC0GTL
	0x01FF	不影响 AD0WINT
	0x0101	
$REF \times (256/4096)$	0x0100	ADC0LTH:ADC0LTL
	0x00FF	AD0WINT=1
	0x0000	
0	0x0000	

给定:

AMX0SL=0x00, AMX0CF=0x00, ADLJST=0,
ADC0LTH:ADC0LTL=0x0200,
ADC0GTH:ADC0GTL=0x0100.

如果 $0x0100 < \text{ADC0 数据字} < 0x0200$, 则 ADC0 转换结束会触发 ADC0 窗口比较中断 (AD0WINT=1)。

给定:

AMX0SL=0x00, AMX0CF=0x00, ADLJST=0,
ADC0LTH:ADC0LTL=0x0100,
ADC0GTH:ADC0GTL=0x0200.

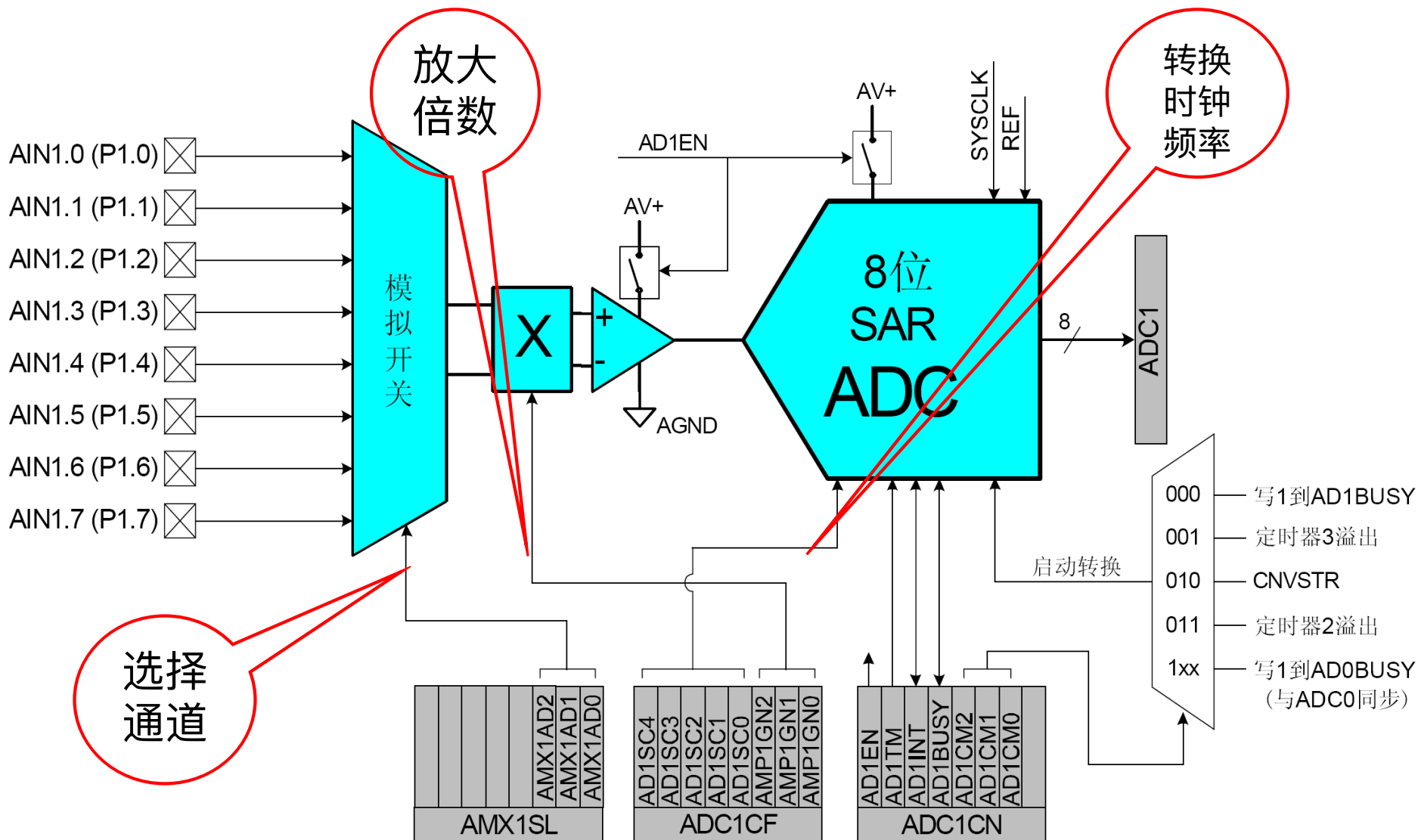
如果 ADC0 数据字 $< 0x0100$ 或 $> 0x0200$, 则 ADC0 转换结束会触发 ADC0 窗口比较中断 (AD0WINT=1)。



- C8051F020还有一个ADC1子系统，包括一个8通道的可配置模拟多路开关（AMUX1），一个可编程增益放大器（PGA1）和一个**500ksps、8位分辨率的逐次逼近寄存器型ADC**，该ADC中集成了跟踪保持电路。
- 与ADC1工作有关的SFR有ADC1配置寄存器**ADC1CF**、AMUX配置寄存器**AMX1SL**、ADC1控制寄存器**ADC1CN**、ADC1数据寄存器**ADC1**。



ADC1 原理框图



• 1. 模拟多路开关和PGA

- 8个通道用于测量，用寄存器AMX1SL选择通道
- PGA对AMUX输出信号的放大倍数由ADC1配置寄存器ADC1CF中的AMP1GN1-0确定
- 增益可用软件编程为0.5, 1, 2, 4

• 2. ADC的工作方式

- ADC1的转换时钟来源于系统时钟分频，由ADC1CF寄存器的AD1SC位决定
- ADC1的转换过程主要由控制寄存器ADC1CN的设置来控制

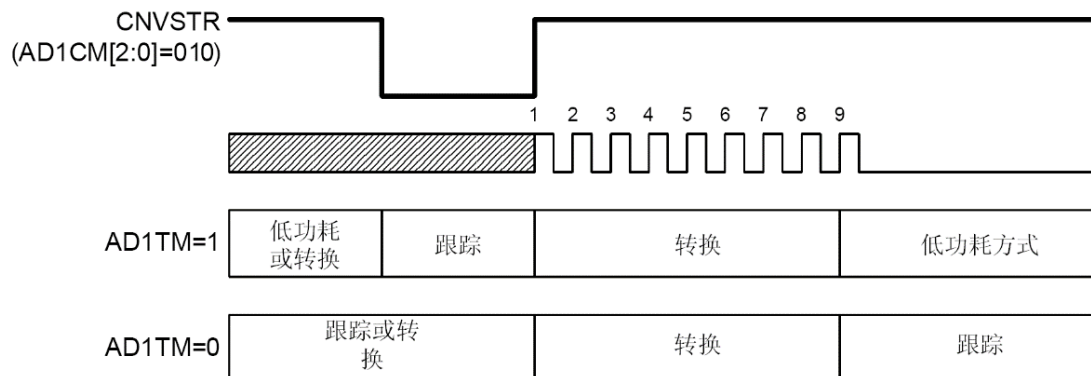
• 3. 跟踪方式

- ADC1CN 中的AD1TM 位控制ADC1 的跟踪保持方式。
- 工作原理与过程同ADC0类似。

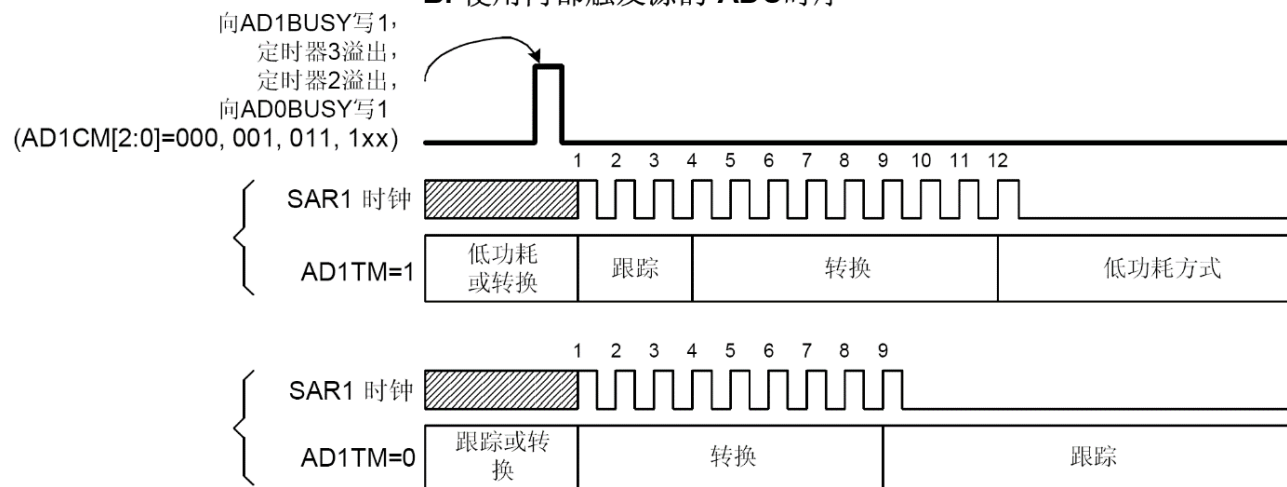


ADC1 跟踪和转换时序举例

A. 使用外部触发源的 ADC时序

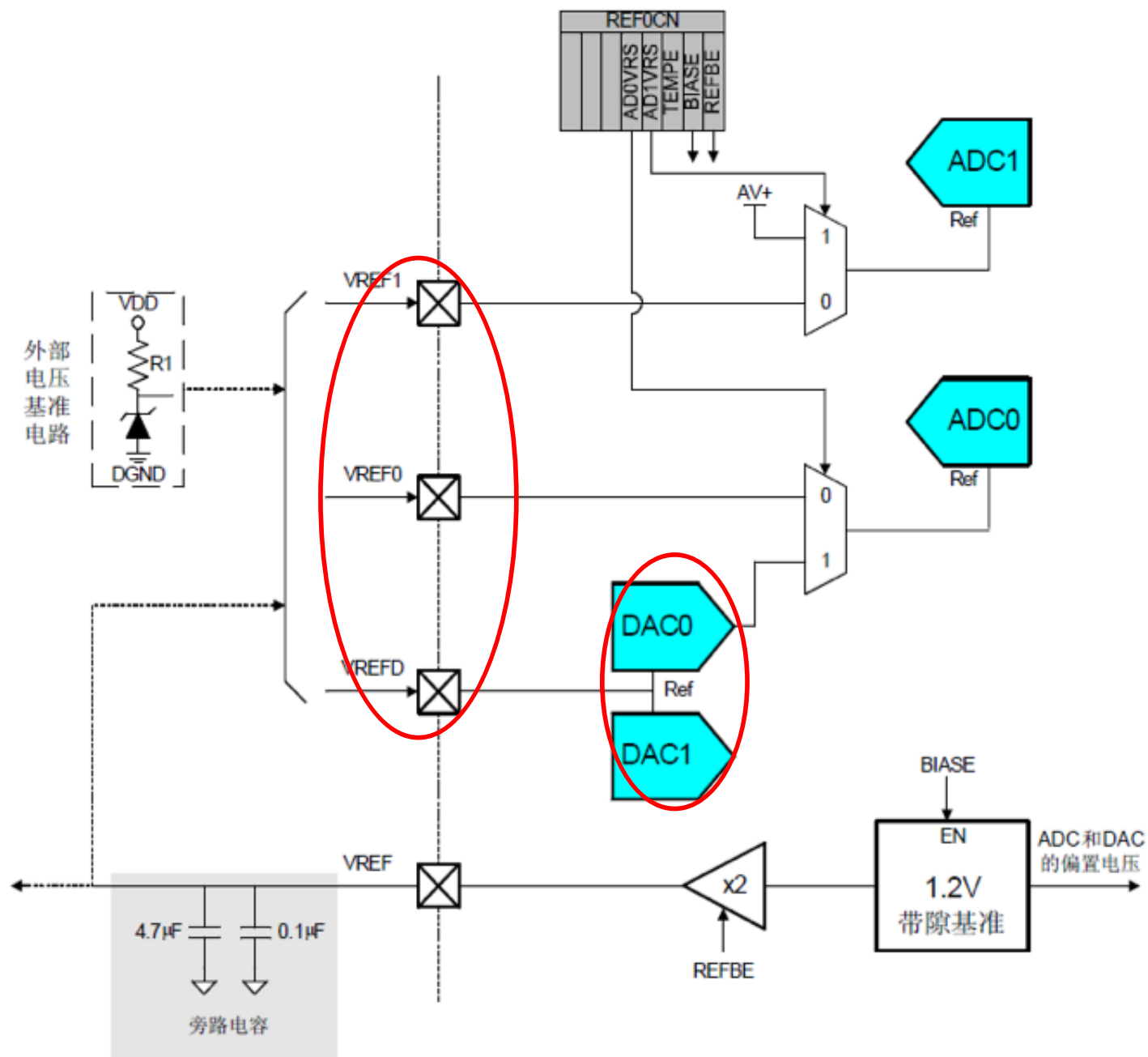


B. 使用内部触发源的 ADC时序



- 电压基准电路为控制ADC 和DAC 模块工作提供了灵活性
- 有三个电压基准输入引脚，允许每个ADC 和两个DAC 使用外部电压基准或片内电压基准。
- 通过配置VREF 模拟开关，ADC0 还可以使用DAC0 的输出作为内部基准，ADC1 可以使用模拟电源电压作为基准。
- 基准电压控制寄存器REF0CN使能/禁止内部基准发生器和选择ADC0、ADC1的基准输入
- 内部电压基准电路由一个1.2V、15ppm/°C（典型值）的带隙电压基准发生器和一个两倍增益的输出缓冲放大器组成。
- 内部电压基准可以通过 V_{REF} 引脚连到应用系统中的外部器件或者电压基准输入引脚





• REF0CN: 电压基准控制寄存器

位7	位6	位5	位4	位3	位2	位1	位0	复位值
-	-	-	AD0VRS	AD1VRS	TEMPE	BIASE	REFBE	0000000
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	SFR地址: 0xD1

位7-5: 未用。读 = 000b, 写 = 忽略。

位4: AD0VRS: ADC0 电压基准选择位

0: ADC0 电压基准取自VREF0 引脚。

1: ADC0 电压基准取自DAC0 输出。

位3: AD1VRS: ADC1 电压基准选择位

0: ADC1 电压基准取自VREF1 引脚。

1: ADC1 电压基准取自AV+。



位7	位6	位5	位4	位3	位2	位1	位0	复位值
-	-	-	AD0VRS	AD1VRS	TEMP E	BIASE	REF BE	0000000
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	SFR地址: 0xD1

位2: TEMPE: 温度传感器使能位

0: 内部温度传感器关闭。

1: 内部温度传感器工作。

位1: BIASE: ADC/DAC 偏压发生器使能位 (使用ADC 和 DAC 时该位必须为1)

0: 内部偏压发生器关闭。

1: 内部偏压发生器工作。

位0: REFBE: 内部电压基准缓冲器使能位

0: 内部电压基准缓冲器关闭。

1: 内部电压基准缓冲器工作。内部电压基准提供从 V_{REF} 引脚输出



• ADC的初始化:

- 配置交叉开关、配置IO端口 → 模拟输入
- 确定基准电压，配置基准寄存器（REF0CN）；
- 配置ADCnCN设置启动方式、跟踪方式和对齐方式
- 配置AMXnCF设置输入方式为单端或差分
- 配置AMXnSL设置通道号
- 确定ADC配置寄存器（ADCnCF）设定ADC转换速度和对模拟量的放大倍数；

• ADC的使用步骤:

- 选择通道
- 启动转换
- 查询是否转换结束
- 等转换结束后把数据转移出来
- 清零转换完成标志位



12.8 模数转换举例

- 1. 片内温度传感器数据采集
- C8051F020的ADC0中有一个片内温度传感器，温度传感器产生一个与器件内部温度成正比的电压。
- 温度转换器的典型应用有系统环境检测、系统过热测试和在基于热电偶的应用中测量冷端温度。
- 本例子使用左对齐，这样可使代码的权值与ADC的位数（12或10）无关。



ADC转换步骤 (8个步骤)

(1) 通过将TEMPE (REF0CN.2) 设置为“1”来允许温度传感器工作。模拟偏置发生器和内部电压基准的允许位REF0CN.1和REF0CN.0置1。

位7	位6	位5	位4	位3	位2	位1	位0	复位值
-	-	-	AD0VR	AD1VR	TEMPE	BIASE	REFBE	0000000
0	0	0	S 0	S 0	1	1	1	

位4: AD0VRS: ADC0 电压基准选择位

0: ADC0 电压基准取自VREF0 引脚。

1: ADC0 电压基准取自DAC0 输出。

位3: AD1VRS: ADC1 电压基准选择位

0: ADC1 电压基准取自VREF1 引脚。

1: ADC1 电压基准取自AV+

位2: TEMPE: 温度传感器使能位

0: 内部温度传感器关闭。

1: 内部温度传感器工作。

位1: BIASE: ADC/DAC 偏压发生器使能位 (使用ADC 和DAC 时该位必须为1)

0: 内部偏压发生器关闭。

1: 内部偏压发生器工作。

位0: REFBE: 内部电压基准缓冲器使能位

0: 内部电压基准缓冲器关闭。

1: 内部电压基准缓冲器工作。内部电压基准提供从VREF 引脚输出

REF0CN|=0x07; //允许温度传感器、模拟偏置发生器和电压基准



- (2) 选择温度传感器作为ADC的输入。这可以通过写AMX0SL来完成
- $AMX0SL = 0x08$; //选择温度传感器作为ADC输入

		AMX0AD3-0								
		0000	0001	0010	0011	0100	0101	0110	0111	1xxx
AMX0CF 位 3-0	0000	AIN0	AIN1	AIN2	AIN3	AIN4	AIN5	AIN6	AIN7	温度传感器
	0001	+(AIN0) -(AIN1)		AIN2	AIN3	AIN4	AIN5	AIN6	AIN7	温度传感器
	0010	AIN0	AIN1	+(AIN2) -(AIN3)		AIN4	AIN5	AIN6	AIN7	温度传感器
	0011	+(AIN0) -(AIN1)		+(AIN2) -(AIN3)		AIN4	AIN5	AIN6	AIN7	温度传感器
	0100	AIN0	AIN1	AIN2	AIN3	+(AIN4) -(AIN5)		AIN6	AIN7	温度传感器
	0101	+(AIN0) -(AIN1)		AIN2	AIN3	+(AIN4) -(AIN5)		AIN6	AIN7	温度传感器
	0110	AIN0	AIN1	+(AIN2) -(AIN3)		+(AIN4) -(AIN5)		AIN6	AIN7	温度传感器
	0111	+(AIN0) -(AIN1)		+(AIN2) -(AIN3)		+(AIN4) -(AIN5)		AIN6	AIN7	温度传感器
	1000	AIN0	AIN1	AIN2	AIN3	AIN4	AIN5	+(AIN6) -(AIN7)		温度传感器
	1001	+(AIN0) -(AIN1)		AIN2	AIN3	AIN4	AIN5	+(AIN6) -(AIN7)		温度传感器
	1010	AIN0	AIN1	+(AIN2) -(AIN3)		AIN4	AIN5	+(AIN6) -(AIN7)		温度传感器



(3) 设置位于ADC0CF中的ADCSAR时钟分频系数，特别是ADC转换时钟的周期至少应为500ns。

(4) 选择ADC的增益。如果使用内部电压基准，则该值大约为2.4V。温度传感器所能产生的最大电压值稍大于1V。因此，可以安全地将ADC的增益设置为“2”，以提高温度分辨率。

ADC0CF=0x41; //设置ADC的时钟为SYSCLK/8. 增益为2

位7	位6	位5	位4	位3	位2	位1	位0	复位值
AD0SC	AD0SC	AD0SC	AD0SC	AD0SC	AMP0GN	AMP0GN1	AMP0GN	11111000
4	3	2	1	0	2		0	
0	1	0	0	0	0	0	1	

位7~3: AD0SC4~0, ADC0SAR转换时钟控制位(ADC转换时钟周期至少为500ns)

$$AD0SC = \frac{SYSCLK}{CLK_{SAR0}} - 1$$

位2~0: AMP0GN2~0, ADC0内部放大器增益 (PGA)

000: 增益=1, 001: 增益=2, 010: 增益=4

011: 增益=8, 10X: 增益=16, 11X: 增益=0.5



(5) 将ADC配置为低功耗跟踪方式.

R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	复位值
AD0EN	AD0TM	AD0INT	AD0BUSY	AD0CM1	AD0CM0	AD0WINT	AD0LJST	00000000
位7	位6	位5	位4	位3	位2	位1	位0	SFR地址:
							(可位寻址)	0xE8

ADC0CN=0xC1 (11000001) ; //允许ADC; 允许低功耗跟踪方式

//清除转换完成标志

//选择ADBUSY作为转换启动源

//清除窗口比较中断

//设置输出数据格式为左对齐



控制寄存器ADC0CN

R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	复位值
AD0EN	AD0TM	AD0INT	AD0BUSY	AD0CM1	AD0CM0	AD0WINT	AD0LJST	00000000
位7	位6	位5	位4	位3	位2	位1	位0	SFR地址: 0xE8 (可位寻址)

位7: AD0EN, ADC0使能位, 0禁止, 1使能

位6: AD0TM, ADC0跟踪方式位。

0: ADC0被使能时, 除了转换期间外一直处于跟踪方式

1: 有AD0CM1~0定义跟踪方式

位5: AD0INT, ADC0转换结束中断标志。该标志必须用软件清0

0: 最后一次清0后, 还没有完成一次数据转换

1: ADC0完成了一次数据转换

位4: AD0BUSY, ADC0忙标志位

读 0: ADC0转换结束或者当前没有正在进行的数据转换

1: ADC0正在进行数据转换

写 0: 无作用, 1: 若AD0CM1,0=00, 启动ADC0转换



控制寄存器ADC0CN

R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	复位值
AD0EN	AD0TM	AD0INT	AD0BUSY	AD0CM1	AD0CM0	AD0WINT	AD0LJST	00000000
位7	位6	位5	位4	位3	位2	位1	位0 (可位寻址)	SFR地址: 0xE8

位3,2: AD0CM1,0, ADC0启动方式选择位

如果AD0TM=0

- 00: 向AD0BUSY写1启动ADC0转换
- 01: 定时器3溢出启动ADC0转换
- 10: CNVSTR上升沿启动ADC0转换
- 11: 定时器2溢出启动ADC0转换

如果AD0TM=1

- 00: 向AD0BUSY写1启动跟踪, 持续3个SAR时钟后进行转换
- 01: 定时器3溢出启动跟踪, 持续3个SAR时钟后进行转换
- 10: 只有当CNVSTR输入为逻辑低电平时ADC0跟踪, 在CNVSTR的上升沿开始转换
- 11: 定时器2溢出启动跟踪, 持续3个SAR时钟后进行转换



(6) 至此，可以通过将AD0BUSY写“1”来启动一次转换：

`AD0BUSY=1;` `//启动转换`

(7) 用查询或中断方式（ADC0CN 的AD0INT位）等待转换完成.

`while (AD0INT ==0);` `//查询方式等待转换结束`

(8) ADC输出寄存器中的值就是与绝对温度成正比的代码。将转换得到的电压值转换成温度值



如何通过采样值得到温度的摄氏度数：

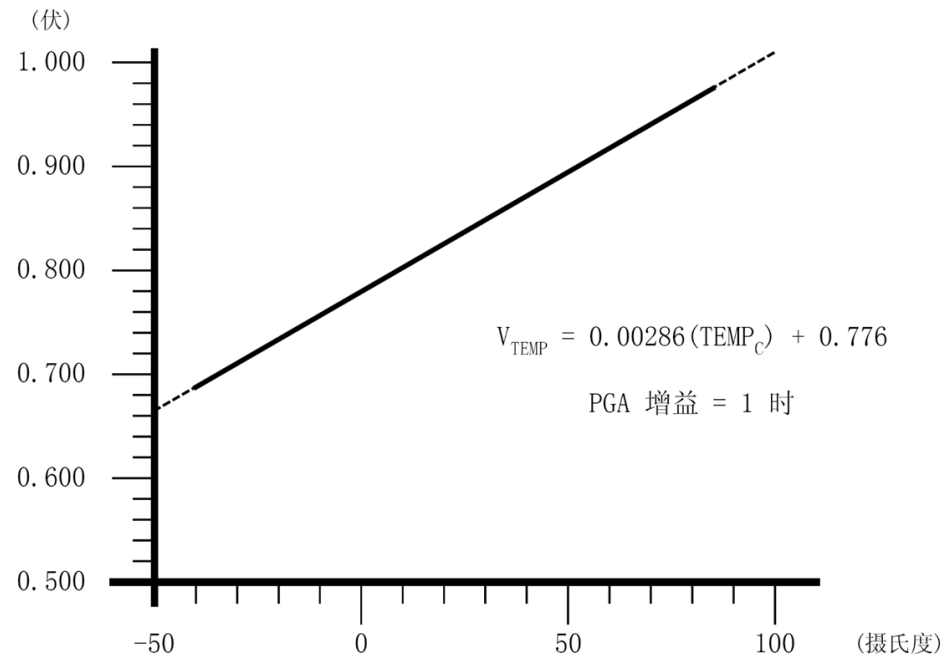
温度传感器产生一个与器件内部绝对温度成正比的电压输出。

$$V_{temp} = (2.86mV / C) \times Temp + 776mV$$

其中 V_{temp} ——温度传感器的输出电压；

$Temp$ ——器件内部的摄氏温度值。

温度传感器的传输特性



采样值为：

$$CODE = V_{in} \times \frac{Gain}{VREF} \times 2^{16}$$

其中 CODE——左对齐的ADC输出代码；

Gain——PGA的增益；

VREF——电压基准的电压值。

假设Gain=2和VREF=2.43V，得到输出温度值为式

$$Temp = \frac{(CODE - 41587)}{154}$$

其中 Temp——温度的摄氏度数。



实际的环境温度测量

- 温度传感器测量的是器件的内部温度。
- 如果希望测量环境温度，则必须考虑器件的自热效应。由于器件功率消耗的原因，测量值很可能比环境温度值高几度。
- 一种方法是在器件上电之后立即启动一次转换，得到一个“冷”温度值；然后大约经过1min之后再测量一次，得到一个“热”温度值。这两个测量值的差就是因自热而产生的温度增加值。
- 另一种方法是让器件从一个低的SYSCLK频率开始工作，例如32kHz，进行一次温度测量，然后再让器件工作在较高频率，例如16MHz的内部振荡器，取两者之差。在时钟频率更低时自热值是可忽略的，因为此时器件的功耗很低。



(1) 查询法程序

- ADC0配置为写AD0BUSY作为转换的开始信号；
- 温度传感器的输出转换成摄氏温度由UART0传输出去；
- 通过PC机超级终端来观察温度采样值。超级终端使用方法为：以Windows2000 系统为例，点击“开始”，进入“程序\附件\通讯目录\”，点击进入超级终端（HyperTerminal）应用程序界面，新建一个通信终端，取名为temp。单击“确定”按钮。选择终端的连接的串口（如串行口1），设置对应于单片通信机程序的通信的格式和协议（如波特率、每字节的位数等）。
- 假设在XTAL1和XTAL2之间连接22.1184MHz晶体
- 系统时钟频率存储在全局常量SYSCLK，目标器件UART波特率存储在全局常量BAUDRATE



主程序

```
#define SYSCLK 22118400
```

```
//系统时钟频率 (Hz)
```

```
#define BAUDRATE 9600
```

```
//UART波特率 (bps)
```

```
void main (void)
```

```
{
```

```
    long temperature;
```

```
// 温度百分之一的精度
```

```
    int temp_int, temp_frac;
```

```
// 温度的整数和小数部分
```

```
    WDTCN = 0xde;
```

```
// 禁止看门狗定时器
```

```
    WDTCN = 0xad;
```

```
    SYSCLK_Init ();
```

```
// 初始化振荡器
```

```
    PORT_Init ();
```

```
// 初始化数据交叉开关和通用IO口
```

```
    UART0_Init ();
```

```
// 初始化UART0
```

```
    ADC0_Init ();
```

```
// 初始化和使能ADC
```



```
while (1)
```

```
{
```

```
    AD0INT = 0;                //清除转换结束标记
```

```
    AD0BUSY = 1;              // 开始转换
```

```
    while (AD0INT == 0);      // 等待转换结束
```

```
    temperature = ADC0;       // 读ADC0数据
```

```
    temperature = temperature - 41857;    // 减去偏移量，对应0°C的值。
```

```
    temperature = (temperature * 100) / 154;
                                     // 计算出对应的温度值 (2.86mV/°C)
```

```
    temp_int = temperature / 100;    // 得到温度值的整数部分
```

```
    temp_frac = temperature % 100;
                                     // 得到温度值的小数部分，2位小数
```

```
    printf ("Temperature is %+02d.%02d\n", temp_int, temp_frac);
```

```
                                     // 从串口输出
```

```
}
```

```
}
```



系统时钟初始化

// 此程序初始化系统时钟使用22.1184MHz晶体作为时钟源

void SYSCLK_Init (void)

{

int i;

OSCXCN = 0x67;

for (i=0; i < 256; i++) ;

while (!(OSCXCN & 0x80)) ;

OSCICN = 0x88;

}

// 延时计数器

// 起动外部振荡器22.1184MHz晶体

// 等待振荡器启动 (>1ms)

// 等待晶体振荡器稳定

// 选择外部振荡器作为系统时钟源

//并使能丢失时钟检测器



IO口初始化

// 配置数据交叉开关和通用IO口

```
void PORT_Init(void)
```

```
{
```

```
    XBR0 = 0x04;
```

```
    XBR1 = 0x00;
```

```
    XBR2 = 0x40;
```

```
    P0MDOUT |= 0x01;
```

```
}
```

// 使能UART0

// 使能数据交叉开关和弱上拉

//使能TX0，推挽输出



UART0初始化

// 配置UART0 使用定时器1产生波特率

```
void UART0_Init(void)
```

```
{
```

```
    SCON0 = 0x50;           // SCON0: 模式1, 8位UART,允许RX
```

```
    TMOD = 0x20;           // TMOD: 定时器1, 模式2, 8位自动重装初值
```

```
    TH1 = -(SYSCLK/BAUDRATE/16); // 按波特率设置定时器1重装值
```

```
    TR1 = 1;                // 起动定时器1
```

```
    CKCON |= 0x10;          // 定时器1使用系统时钟为时基
```

```
    PCON |= 0x80;          // SMOD=1
```

```
    TI0 = 1;                // 表示就绪
```

```
}
```



ADC0初始化

// 配置ADC0 使用AD0BUSY作为转换源, 使用左对齐输出模式,

// 使用正常跟踪模式, 测量片内温度传感器器输出

// 禁止ADC0转换结束中断和ADC0窗口比较器中断

```
void ADC0_Init (void)
```

```
{
```

```
    ADC0CN = 0x81;           // ADC0使能;正常跟踪模式
```

```
                                // 当写AD0BUSY时ADC0转换开始ADC0数据左对齐
```

```
    REF0CN = 0x07;           // 使能温度传感器片内VREF和VREF输出缓冲器
```

```
    AMX0SL = 0x0f;           // 选择温度传感器作为ADC多路模拟转换器输出
```

```
    ADC0CF = (SYSCLK/2500000) << 3;    // ADC转换时钟=2.5MHz
```

```
    ADC0CF |= 0x01;           // PGA增益=2
```

```
    EIE2 &= ~0x02;           // 禁止ADC0 EOC中断
```

```
    EIE1 &= ~0x04;           // 禁止ADC0窗口比较器中断
```

禁止
关闭

```
}
```



(2) 中断方式程序

- 在中断模式使用定时器3溢出作为转换开始信号，测量片内温度传感器输出。
- ADC0结果经简单的均值滤波处理，均值滤波计数值由常量INT_DEC给出。
- ADC结果经计算得出温度从UART0传输。
- 假设在XTAL1和XTAL2之连接22.1184MHz晶体。
- 系统时钟频率存储在全局常量SYSCLK 目标UART波特率存储在全局常量BAUDRATE。
- ADC0采样率存储在全局常量SAMPLERATE0。



```
#define SYSCLK 22118400           //系统时钟频率 (Hz)
#define BAUDRATE 9600            //UART波特率 (bps)
#define SAMPLERATE0 50000        //ADC0采样频率 (Hz)
#define INT_DEC 256              //均值滤波计数值
```

```
void main(void)
```

```
{
```

```
    long temperature;           //精度为百分之一的温度
```

```
    int temp_int,temp_frac; //温度的整数和小数部分
```

```
    WDTCN = 0xde;               //禁止开门狗定时器
```

```
    WDTCN = 0xad;
```

```
    SYSCLK_Init();              //初始化振荡器
```

```
    PORT_Init();                //初始化数据交叉开关和通用IO接口 a为引脚
```

```
    UART0_init();               //初始化UART0 samplerate0 ~ sysclk * (2^16 - a)
```

```
    Timer3_Init (SYSCLK/SAMPLERATE0); //初始化定时器3溢出为采样速率
```

```
    ADC0_Init ();               // 初始化ADC
```

```
    AD0EN = 1;                  // 使能ADC
```

```
    EA = 1;                     // 使能所有中断
```




```
while (1)
```

```
{  
    EA = 0;                // 禁止中断  
    temperature = result;  // result是经数字滤波后的结果  
    EA = 1;                // 重使能中断  
    temperature = temperature - 41380;    // 计算温度百分之一精度  
    temperature = (temperature * 100) / 156;  
    temp_int = temperature / 100;  
    temp_frac = temperature % 100;  
    printf ("Temperature is %+02d.%+02d\n", temp_int, temp_frac);  
}  
}
```



系统时钟初始化

// 此程序初始化系统时钟使用22.1184MHz晶体作为时钟源

void SYSCLK_Init (void)

{

int i;

OSCXCN = 0x67;

for (i=0; i < 256; i++) ;

while (!(OSCXCN & 0x80)) ;

OSCICN = 0x88;

}

// 延时计数器

// 起动外部振荡器22.1184MHz晶体

// 等待振荡器启动 (>1ms)

// 等待晶体振荡器稳定

// 选择外部振荡器作为系统时钟源

//并使能丢失时钟检测器



IO口初始化

// 配置数据交叉开关和通用IO口

```
void PORT_Init(void)
```

```
{
```

```
    XBR0 = 0x04;
```

```
    XBR1 = 0x00;
```

```
    XBR2 = 0x40;
```

```
    P0MDOUT |= 0x01;
```

```
}
```

// 使能UART0

// 使能数据交叉开关和弱上拉

//使能TX0，推挽输出



UART0初始化

// 配置UART0 使用定时器1产生波特率

```
void UART0_Init(void)
```

```
{
```

```
    SCON0 = 0x50;           // SCON0: 模式1, 8位UART,允许RX
```

```
    TMOD = 0x20;           // TMOD: 定时器1, 模式2, 8位自动重装初值
```

```
    TH1 = -(SYSCLK/BAUDRATE/16); // 按波特率设置定时器1重装值
```

```
    TR1 = 1;                // 起动定时器1
```

```
    CKCON |= 0x10;          // 定时器1使用系统时钟为时基
```

```
    PCON |= 0x80;           // SMOD=1
```

```
    TI0 = 1;                // 表示就绪
```

```
}
```



ADC0初始化

配置ADC0 使用定时器3溢出作为转换源, 转换结束产生中断

使用左对齐输出模式允许ADC转换结束中断, 不使用时禁止ADC

```
void ADC0_Init (void)
```

```
{  
    ADC0CN = 0x05;           // ADC0禁止; 正常跟踪 mode;  
                               // 定时器3溢出ADC0转换开始  
                               // ADC0数据是左对齐  
    REF0CN = 0x07;           // 允许温度传感器片内VREF和VREF输出缓冲器  
    AMX0SL = 0x0f;           // 选择温度传感器作为ADC多路模拟转换输出  
    ADC0CF = (SYSCLK/2500000) << 3;  
                               // ADC转换时钟=2.5MHz  
    ADC0CF |= 0x01;           // PGA增益=2  
    EIE2 |= 0x02;             // 允许ADC中断  
}
```



定时器3初始化

配置定时器3,自动重装间隔由“counts”指定,不产生中断,使用系统时钟为时基.

```
void Timer3_Init (int counts)
{
    TMR3CN = 0x02;           // 停止定时器3; 清除 TF3;
                              // 使用系统时钟为时基
    TMR3RL = -counts;        // 初始化重装值
    TMR3 = 0xffff;           // 设置为立即重装
    EIE2 &= ~0x01;          // 禁止定时器3中断
    TMR3CN |= 0x04;          // 起动定时器3
}
```



中断服务程序

得到ADC0采样值，将它加到运行总数<accumulator>中，数字滤波计数器<int_dec>减1，当<int_dec>为0时，在全局变量<result>放置数字滤波后的结果

```
void ADC0_ISR (void) interrupt 15
```

```
{  
    static unsigned int_dec=INT_DEC;    //数字滤波计数器，int_dec = 0时重设新值256  
    static long accumulator=0L;  
    AD0INT = 0;                        // 清除ADC转换结束标志  
    accumulator += ADC0;                // 读ADC值并加到运行总数中  
    int_dec--;                          // 更新数字滤波计数器  
    if (int_dec == 0)                  // 如果为0，计算结果  
    {  
        int_dec = INT_DEC;            // 重设计数器  
        result = accumulator >> 8;    // 除以256，求平均值（数字滤波）  
        accumulator = 0L;              // 复位accumulator  
    }  
}
```



2. 多通道数据采集

- **// 此程序为ADC0的应用例程在中断模式使用定时器3溢出作为开始转换信号**
- **// 测量AIN0到AIN7的电压和温度传感器**
- **// 转换结果经过计算所得电压从UART0传输**
- **// 假设在XTAL1和XTAL2之间接22.1184MHz晶体**
- **// 系统时钟频率存储在全局常量SYSCLK.**
- **//目标UART波特率存储在全局常量BAUDRATE.**
- **// ADC0采样频率存储在全局常量SAMPLERATE0.**
- **//电压参考值存储在VREF0**




```
#define SYSCLK 22118400           //系统时钟频率 (Hz)
#define BAUDRATE 9600            //UART波特率 (bps)
#define SAMPLERATE0 50000        //ADC0采样频率 (Hz)
#define VREF0 2430               //VREF参考电压 (mV)

void main (void)
{
    long voltage;                // 电压以mV为单位
    int i;                       // 循环计数器
    WDTCN = 0xde;                // 禁止看门狗定时器
    WDTCN = 0xad;
    SYSCLK_Init ();              // 初始化振荡器
    PORT_Init ();                // 初始化数据交叉开关和通用IO
    UART0_Init ();               // 初始化UART0
    Timer3_Init (SYSCLK/SAMPLERATE0); // 初始化定时器3溢出作为采样率
    ADC0_Init ();                // 初始化ADC
    AD0EN = 1;                   // 允许ADC
```



```
while (1)
{
    for (i = 0; i < 9; i++)
    {
        EA = 0;                // 禁止中断
        voltage = result[i];    // 从全局变量取得ADC值
                                // results[i]存储AIN0-7和温度传感器输出结果
        EA = 1;                // 重新使能中断
                                // 计算电压mV

        voltage = voltage * VREF0;
        voltage = voltage >> 16;
        printf ("Channel '%d' voltage is %ldmV\n", i, voltage);
    }
}
```



系统时钟初始化

// 此程序初始化系统时钟使用22.1184MHz晶体作为系统时钟

```
void SYSCLK_Init (void)
```

```
{
```

```
    int i;
```

```
    OSCXCN = 0x67;
```

```
    for (i=0; i < 256; i++) ;
```

```
    while (!(OSCXCN & 0x80)) ;
```

```
    OSCICN = 0x88;
```

```
}
```

// 延时计数器

// 起动外部振荡器22.1184MHz晶体

// 等待振荡器启动 (>1ms)

// 等待晶体振荡器稳定

// 选择外部振荡器作为系统时钟

//源并允许丢失时钟检测器



IO口初始化

// 配置数据交叉开关和通用IO口

```
void PORT_Init(void)
```

```
{
```

```
    XBR0 = 0x04;
```

```
    XBR1 = 0x00;
```

```
    XBR2 = 0x40;
```

```
}
```

// 使能UART0

// 使能数据交叉开关和弱上拉



UART0初始化

// 配置UART0 使用定时1为波特率发生器

```
void UART0_Init(void)
```

```
{
```

```
    SCON0 = 0x50;
```

```
// SCON0: 模式1, 8位UART, 允许RX
```

```
    TMOD = 0x20;
```

```
// TMOD: 定时器1, 模式2, 8位重装
```

```
    TH1 = -(SYSCLK/BAUDRATE/16);
```

```
// 按波特率设置定时器1重装值
```

```
    TR1 = 1;
```

```
// 启动定时器1
```

```
    CKCON |= 0x10;
```

```
// 定时器1使用系统时钟为时基
```

```
    PCON |= 0x80;
```

```
// SMOD00 = 1
```

```
    TI0 = 1;
```

```
// 表示TX0就绪
```

```
}
```



ADC0初始化

// 配置ADC0 使用定时器3 溢出作为转换源, 转换结束产生中断使用左对齐输出模式

// 使能ADC转换结束中断禁止ADC

//注意: 使能低功率跟踪模式保证当改变通道时的跟踪次数最少

void ADC0_Init (void)

{

ADC0CN = 0x45;

// ADC0 禁止; 低功率跟踪模式

// 当定时器3溢出时ADC0转换开始;

// ADC0数据左对齐

REF0CN = 0x07;

// 使能温度传感器片内VREF和

//VREF输出缓冲器

AMX0SL = 0x00;

// 选择AIN0为ADC多路模拟输出

ADC0CF = (SYSCLK/2500000) << 3; // ADC转换时钟=2.5MHz

ADC0CF &= ~0x07;

// PGA增益=1

EIE2 |= 0x02;

// 允许ADC中断

}



定时器3初始化

// 配置定时器3,自动重装间隔由“counts”指定，不产生中断，使用系统时钟为时基。

```
void Timer3_Init (int counts)
```

```
{
```

```
    TMR3CN = 0x02;
```

```
// 停止定时器3; 清除 TF3;
```

```
// 使用系统时钟为时基
```

```
    TMR3RL = -counts;
```

```
// 初始化重装值
```

```
    TMR3 = 0xffff;
```

```
// 设置为立即重装
```

```
    EIE2 &= ~0x01;
```

```
// 禁止定时器3中断
```

```
    TMR3CN |= 0x04;
```

```
// 起动定时器3
```

```
}
```



中断服务程序

// ADC0转换结束中断服务程序

// 得当ADC0采样值并存储在全局数组 <result>.

// 同时选择下一个通道转换

void ADC0_ISR (void) interrupt 15

```
{  
    → 只赋一次初值.  
    static unsigned char channel = 0;  
    AD0INT = 0;  
    result[channel] = ADC0;  
    channel++;  
    if (channel == 9)  
    {  
        channel = 0;  
    }  
    AMX0SL = channel;  
}
```

// ADC多路模拟通道(0-8)

// 清除ADC转换结束标志

// 读ADC值

// 改变通道

// 设置多路模拟转换器到下一个通道

